BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is the set of all distinct situations the world can be in and how they connect. In here, it's every walkable grid cell and its edges to neighbours. It's finite and each state appears once. The search tree is what the algorithm builds while exploring it. So the same state can appear many times as different tree nodes (reached by different routes). So, the state space is the fixed map (a graph) and the search tree is the dynamic record of how the algorithm walked that map.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It prevents re-expanding states already seen, which does two things. It stops infinite loops caused by cycles and it prunes redundant work so the search terminates. Without it, DFS would follow a cycle down forever, pushing the same cells repeatedly and never terminating on any graph with a loop which every grid is. The reached set is exactly what converts tree search into graph search.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In this grid every move costs the same, so "shortest path" = "fewest edges." BFS expands strictly in order of increasing depth, so the first time it reaches the goal it has done so by the fewest possible moves guaranteed optimal when all step costs are equal. DFS instead dives as deep as possible down one branch before backtracking, so it commits to whatever direction it happens to expand first and only stops when it stumbles onto the goal. It returns the first path found, not the shortest. UCS matches BFS here because with uniform cost, ordering by g(n) is the same as ordering by depth.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

This is the crucial BFS/DFS trade-off. With branching factor b and solution depth d - BFS space is O(b^d). It must hold the entire frontier in memory at once, and that frontier grows exponentially with depth. On a huge grid the frontier can balloon to hundreds of thousands of cells before the goal is found, exhausting RAM. DFS space is only O(b·m). It only stores the nodes along the current path plus their siblings (m = max depth), which is linear, not exponential. So DFS is far cheaper on memory. The trade-off -> DFS buys that low memory by giving up optimality. UCS has the same exponential-space bottleneck as BFS because it too keeps a large frontier. So on the 1000×1000 grid, BFS/UCS die from memory, while DFS survives on memory but returns a bad path.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

FACULTY OF COMPUTING